

Custom Date Picker Documentation

Overview

This documentation explains a **custom Date Picker function** in Flutter that: - Opens a date selection dialog - Applies a **custom theme** (colors & background) - Formats the selected date - Displays it inside a `TextEditingController`

Purpose

- Select date of birth
 - Pick appointment or schedule date
 - Show formatted date in a text field
-

Function Code

```
Future<void> pickDate(BuildContext context) async {
  DateTime? selectedDate = await showDatePicker(
    context: context,
    initialDate: DateTime.now(),
    firstDate: DateTime(1950),
    lastDate: DateTime(2050),
    builder: (context, child) {
      return Theme(
        data: Theme.of(context).copyWith(
          dialogTheme: const DialogThemeData(
            backgroundColor: AppColors.primaryColor,
          ),
          colorScheme: const ColorScheme.light(
            primary: AppColors.buttonColor,
            onPrimary: AppColors.primaryColor,
            onSurface: AppColors.c25324B,
          ),
        ),
        child: child!,
      );
    },
  );

  if (selectedDate != null) {
    setState(() {
```

```

        dateController.text = "${selectedDate.day.toString().padLeft(2, '0')}/${
            "${selectedDate.month.toString().padLeft(2, '0')}/${
                "${selectedDate.year}";
    });
}
}

```

Step-by-Step Explanation

1. `showDatePicker`

Opens the system date picker dialog.

```
initialDate: DateTime.now(),
```

- Default selected date is **today**

```
firstDate: DateTime(1950),
lastDate: DateTime(2050),
```

- Limits selectable date range

2. Custom Theme (UI Styling)

```
builder: (context, child) {
  return Theme(
```

This allows you to **customize the date picker UI**.

Applied Customizations

- Dialog background color
- Primary button color
- Text color

```
dialogTheme: DialogThemeData(
  backgroundColor: AppColors.primaryColor,
),
```

```
colorScheme: ColorScheme.light(  
    primary: AppColors.buttonColor,  
    onPrimary: AppColors.primaryColor,  
    onSurface: AppColors.c25324B,  
),
```

3. Handling Selected Date

```
if (selectedDate != null)
```

- Ensures user did not cancel the picker

4. Date Formatting

```
"${selectedDate.day.toString().padLeft(2, '0')}/"  
"${selectedDate.month.toString().padLeft(2, '0')}/"  
"${selectedDate.year}";
```

Output Format

```
DD/MM/YYYY
```

Example:

```
05/09/2025
```

5. Updating UI

```
setState(() {  
    dateController.text = formattedDate;  
});
```

- Updates text field instantly
- Reflects selected date in UI

Required Variables

```
TextEditingController dateController = TextEditingController();
```

Usage Example

```
TextField(  
  controller: dateController,  
  readOnly: true,  
  onTap: () => pickDate(context),  
  decoration: const InputDecoration(  
    hintText: 'Select date',  
  ),  
);
```

Key Benefits

- Clean and reusable date picker logic
 - Custom UI consistent with app theme
 - Prevents invalid date selection
 - User-friendly formatted output
-

Notes

- Can be reused for any date field
 - Date range can be adjusted easily
 - Supports localization if needed
-

Summary

This function provides a **custom-themed, reusable, and well-formatted Date Picker** solution in Flutter that cleanly updates UI using `TextEditingController`.

You can freely edit this documentation as needed.